

# App La Bailoteca

Antonio Manuel Aragón Pérez – 2DAM

Repositorio GitHub: <https://github.com/TonyArPe/ProyectoBailoteca.git>

Bienvenido al proyecto de la **App de la Bailoteca**. Este es un proyecto desarrollado en **Android Studio** para la gestión de profesores de una academia de baile. A continuación, se detallan las funcionalidades de la app y cómo funciona.

## Funcionalidades principales

### 1. Inicio de sesión simple

- Al iniciar la aplicación, te da la bienvenida con tu nombre de usuario si ya iniciaste sesión anteriormente.
- Los datos del usuario se guardan usando **SharedPreferences**, lo que permite que se mantengan hasta que desinstales la app o borres los datos.

### 2. Lista de Profesores

- La app muestra una lista de profesores en forma de tarjetas (Cards) usando un **RecyclerView**.
- Cada tarjeta incluye:
  - Una **imagen** del profesor.
  - Su **nombre**.
  - Su **especialidad**.
  - Valoración mediante un **RatingBar**.
  - Un **botón de contacto** para enviar un correo directamente al profesor.
- 3. **Botones flotantes (FAB)** En la esquina inferior derecha, tienes tres botones flotantes con estas funciones:

#### a) Añadir Profesor

- Al pulsar este botón, aparece un formulario donde puedes:
- Escribir el **nombre** del profesor.
- Especificar su **especialidad** (por ejemplo: "Salsa", "Bachata").
- Indicar si es "**destacado**" o no, marcando un **checkbox**.
- Al confirmar, el nuevo profesor se agrega a la lista y se muestra al

#### instante. b) Actualizar Profesor

- Este botón te permite actualizar los datos de un profesor ya existente.
- Aparece un formulario donde puedes:
- Indicar la **posición** del profesor que quieres editar (ejemplo: si es el primer profesor, posición = 0).
- Cambiar el **nombre**, la **especialidad** y si es **destacado** o no.
- Si la posición es válida, la información del profesor se actualiza en la

#### lista. c) Eliminar Profesor

- Este botón te permite eliminar un profesor de la lista.

- Aparece un **diálogo** donde ingresas la **posición** del profesor que quieres eliminar.
- Si la posición es válida, el profesor desaparece de la lista.

#### 4. Perfil de Profesor

- Al seleccionar un profesor de la lista, accedes a su **perfil detallado**, donde puedes ver:
  - **Nombre.**
  - **Especialidad.**
  - **Descripción.**
  - **Correo electrónico.**
  - **Imagen** del profesor.
- Además, puedes **contactar** con el profesor a través de un botón de correo, el cual abre el cliente de correo electrónico para enviar un mensaje predefinido.

#### 5. Botón de Contactar

- En el perfil de cada profesor, se encuentra un botón de **contactar**, el cual permite enviar un correo electrónico al profesor directamente desde la app.
- El correo electrónico se envía con un asunto predefinido (consulta sobre clases) y un mensaje que incluye su nombre y especialidad.

#### 6. Diseño Atractivo

- El diseño de la app utiliza una paleta de colores **morados y lilas**, creando una interfaz moderna y agradable para el usuario.
- Se utiliza **CardView** para mostrar la lista de profesores, lo que da un aspecto limpio y organizado.
- Los botones están diseñados con colores vibrantes, haciendo que la interacción sea más intuitiva.

#### 7. Gestión de Profesores en la Pantalla Principal

- Los profesores son gestionados desde la pantalla principal con una vista **RecyclerView** que muestra todos los datos en formato tarjeta.
- Cada tarjeta tiene un botón para **expandir** y ver más detalles, así como los botones de **editar** y **eliminar** para gestionar la información del profesor de manera rápida.

---

## Tecnologías Usadas

- **Kotlin:** Lenguaje de programación utilizado para desarrollar la aplicación.
  - **Android Studio:** IDE utilizado para desarrollar la aplicación.
  - **RecyclerView:** Para mostrar la lista de profesores de manera eficiente.
  - **Glide:** Para cargar imágenes de los profesores de forma rápida y eficiente.
  - **SharedPreferences:** Para almacenar los datos del usuario y mantener la sesión activa.
  - **Intent:** Para enviar correos electrónicos a los profesores de forma directa desde la app.
  - **CardView:** Para mostrar la lista de profesores con un diseño atractivo.
-

# Cómo usar la aplicación

## 1. Inicio de sesión:

- Al abrir la app por primera vez, se te pedirá que inicies sesión. Si ya has iniciado sesión previamente, serás recibido por tu nombre de usuario.

## 2. Ver Profesores:

- En la pantalla principal verás una lista de profesores. Puedes hacer clic en cualquier profesor para ver más detalles en su perfil.

## 3. Añadir, editar o eliminar un profesor:

- Usa los botones flotantes para agregar un nuevo profesor, actualizar los datos de uno existente o eliminar un profesor de la lista.

## 4. Contactar con el Profesor:

- En el perfil de cada profesor, haz clic en el botón "**Contactar**" para enviar un correo al profesor directamente desde la aplicación.

## VERSION 1.3

### Documentación de Firebase:

[Documentación básica de Firebase](#)

---

### Integración y Mejoras Recientes

#### Firestore Configurado

Se configuró **Firestore** para autenticar usuarios y manejar los datos de manera eficiente. Los pasos clave fueron:

1. Crear un proyecto en la consola de Firebase.
2. Descargar y añadir el archivo **google-services.json** al proyecto.
3. Configurar la huella digital **SHA-1** en Firebase para habilitar la autenticación segura.
4. Habilitar el método de autenticación por **correo electrónico y contraseña** en la consola de Firebase.

#### Mejoras en el Código

#### Actividad de Registro

- Ajustamos el código para validar que todos los campos del formulario estén correctamente completados.

- Implementamos mensajes claros para el usuario en caso de errores (como contraseñas débiles o formatos de correo incorrectos).

### **Actividad de Inicio de Sesión**

- Se mejoró la gestión de errores, mostrando mensajes más detallados cuando el inicio de sesión falla.
- Validación de entradas como correos en blanco o contraseñas incorrectas.
- Configuración de nuestro proyecto en Firebase para la autenticación por correo y contraseña

### **Mejora Visual**

#### **Pantalla de Inicio de Sesión y Registro**

- Se añadieron mejores diseños.
- Botones con colores acordes a la temática y el logo y bordes redondeados.
- Fondos personalizados para mejorar la estética.
- Todos los elementos visuales fueron adaptados para mantener coherencia con la temática general de la app.

### **Pruebas y Resultados**

- **Registro y Autenticación:** Se probaron múltiples casos, como intentos de registro con correos no válidos, contraseñas débiles y campos vacíos, para garantizar el correcto funcionamiento.
- **Persistencia de Sesión:** Confirmamos que, al cerrar y abrir la app, la sesión se mantiene activa.
- **Interfaz Mejorada De Forma Responsiva**

## **VERSION 2.1**

### **Implementación de MVVM y Clean Architecture**

#### **¿Por qué MVVM y Clean Architecture?**

- **MVVM (Model-View-ViewModel):** Separa la lógica de la UI, facilitando la gestión de datos y la actualización de la interfaz de usuario.
- **Clean Architecture:** Divide el proyecto en capas independientes (presentación, dominio y datos), lo que permite un código más modular y fácil de mantener.

#### **Capas de Clean Architecture**

1. **Capa de Presentación:** Contiene los fragments y ViewModels.
2. **Capa de Dominio:** Define la lógica de negocio (casos de uso) y las interfaces de repositorios.
3. **Capa de Datos:** Implementa los repositorios y gestiona las fuentes de datos (API, base de datos, etc.).

---

## **Pasos Realizados**

### **1. Creación de las Entidades del Dominio**

Se definieron las entidades Event, Professor y Video en el paquete domain. Estas entidades representan los datos principales de la aplicación.

```
data class Professor(
    val imageResId: Int,
    val name: String,
    val specialty: String,
    val isTopRated: Boolean,
    val description: String,
    val email: String
)
```

## **2. Definición de los Repositorios**

Se crearon interfaces en el paquete domain para definir cómo se accederá a los datos.

```
interface ProfessorRepository {
    suspend fun getProfessors(): List<Professor>
    suspend fun saveProfessor(professor: Professor)
}
```

## **3. Implementación de los Repositorios**

En el paquete data, se implementaron las interfaces de los repositorios. Aquí se gestiona la obtención de datos desde fuentes externas (API, base de datos, etc.).

```
class ProfessorRepositoryImpl : ProfessorRepository {
    override suspend fun getProfessors(): List<Professor> {
        // Lógica para obtener profesores
    }
}
```

## **4. Creación de los Casos de Uso**

Los casos de uso encapsulan la lógica de negocio. Por ejemplo, GetProfessorsUseCase se encarga de obtener la lista de profesores.

```
class GetProfessorsUseCase(private val professorRepository: ProfessorRepository) {
    suspend operator fun invoke(): List<Professor> {
        return professorRepository.getProfessors()
    }
}
```

## **4. Desarrollo de los ViewModels**

Los ViewModels preparan los datos para la UI. Por ejemplo, ProfessorViewModel obtiene la lista de profesores y la expone a la UI.

```

class ProfessorViewModel(private val getProfessorsUseCase:
GetProfessorsUseCase) : ViewModel() {
    private val _professors = MutableStateFlow<List<Professor>>(emptyList())
    val professors: StateFlow<List<Professor>> get() = _professors

    fun loadProfessors() {
        viewModelScope.launch {
            _professors.value = getProfessorsUseCase()
        }
    }
}

```

## 5. Integración con la UI

Los fragments (EventFragment, ProfessorFragment, VideoFragment) utilizan los ViewModels para obtener los datos y mostrarlos en la UI.

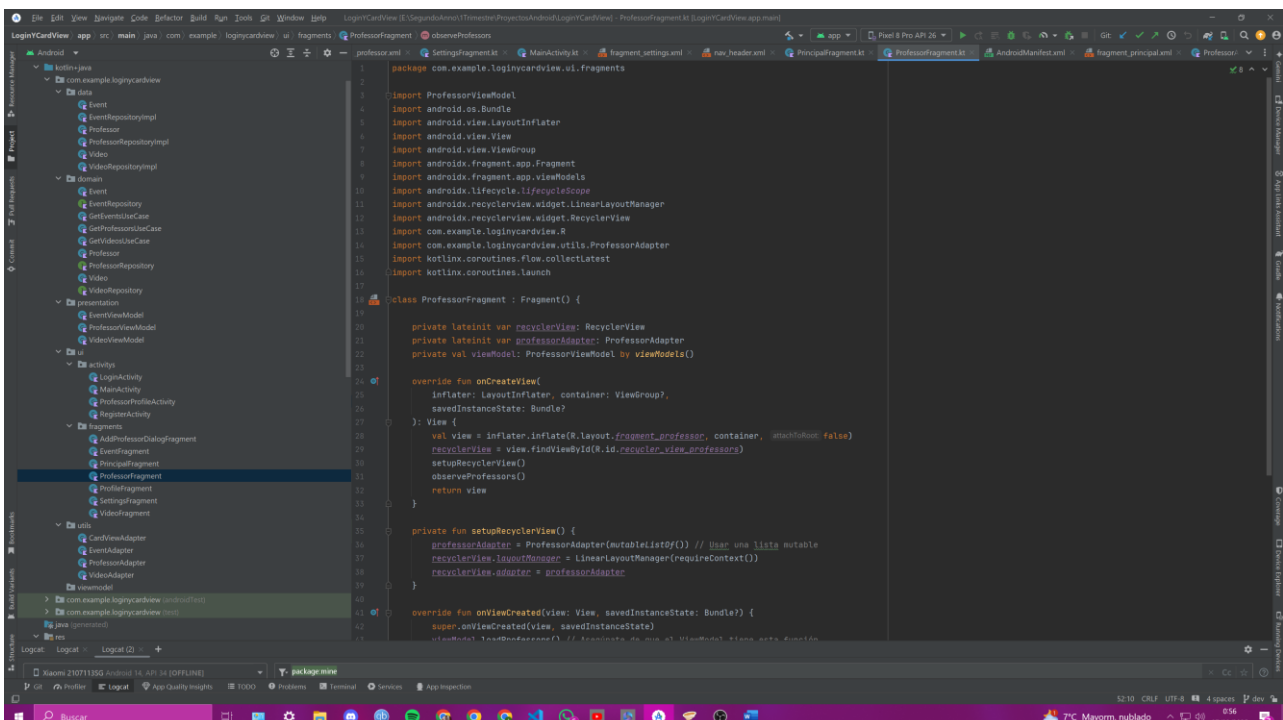
```

class ProfessorFragment : Fragment() {
    private val viewModel: ProfessorViewModel by viewModels()

    override fun onCreateView(view: View, savedInstanceState: Bundle?) {
        super.onCreateView(view, savedInstanceState)
        viewModel.loadProfessors()
        viewModel.professors.observe(viewLifecycleOwner) { professors ->
            // Actualizar la UI con la lista de profesores
        }
    }
}

```

## PANTALLAZO DE LA ESTRUCTURA DE CARPETAS DE MI PROYECTO



## **VERSION 3.1 Y 3.2**

### **Características Implementadas**

1. **Captura de Imagen desde la Cámara**
  - Los usuarios pueden tomar fotos directamente desde la cámara de su dispositivo para utilizarlas en la aplicación.
2. **Selección de Imagen desde la Galería**
  - Además de la captura de imágenes, los usuarios pueden seleccionar imágenes desde la galería de su dispositivo.
3. **Permisos en Tiempo de Ejecución**
  - Se solicitan permisos en tiempo de ejecución para el acceso a la cámara y al almacenamiento, garantizando el cumplimiento de las políticas de privacidad y seguridad.
4. **Conversión de Imágenes a Base64**
  - Se preparó un método para convertir imágenes en formato bitmap a Base64, facilitando su manipulación y almacenamiento para futuras implementaciones.
5. **Visualización en DialogFragment**
  - Se incluye un ImageView en el DialogFragment de creación/modificación de ítems para mostrar la imagen capturada o seleccionada.

### **Configuración Técnica**

- Se configuró un FileProvider en el archivo AndroidManifest.xml para manejar URIs de archivos de forma segura, evitando conflictos con permisos de acceso a archivos post Android N.
- Se definieron las rutas accesibles para el FileProvider en el archivo file\_paths.xml bajo el directorio res/xml/, asegurando que la aplicación pueda acceder a las imágenes almacenadas de forma temporal o permanente.

### **Uso**

Para utilizar estas funciones, el usuario debe interactuar con los elementos de UI específicos que desencadenan la captura o selección de imágenes. Estas opciones están accesibles a través de botones dedicados en el DialogFragment de modificación de ítems.

### **Seguridad y Privacidad**

- Todos los accesos a hardware y almacenamiento externo están regulados por solicitudes de permisos explícitas, conforme a las directrices y mejores prácticas de seguridad de Android.

### **Posicionamiento Opcional**

- Se deja como opcional la versión 3.2, que incluiría funcionalidades de geolocalización de ítems usando Google Maps, para aquellos usuarios que deseen añadir contexto geográfico a sus imágenes o ítems.

## **VERSION 4.1**

### **1. Configuración Inicial de Firebase**

Antes de poder interactuar con Firestore en nuestra aplicación, debemos configurar **Firebase en Android**.

#### **1.1. Crear un proyecto en Firebase**

1. Ve a Firebase Console
2. Crea un **nuevo proyecto**.
3. **Añade Firebase a tu aplicación Android:**
  - Introduce el nombre del paquete de tu aplicación (Ej: com.example.loginycardview)
  - Descarga el archivo google-services.json y colócalo en:

app/google-services.json

### **2.1. Agregar Firebase a build.gradle**

En build.gradle (Project), dentro de dependencies, añade:

```
classpath 'com.google.gms:google-services:4.3.10'
```

En build.gradle (Module: app), agrega:

```
// Plugins
plugins {
    id 'com.google.gms.google-services'
}

// Dependencias de Firebase
dependencies {
    implementation 'com.google.firebase:firebase-firestore-ktx'
}
```

**Guarda y sincroniza el proyecto.**

---

## **2. Configuración de Firestore**

Para trabajar con Firestore, necesitamos **crear la base de datos y definir reglas de seguridad**.

### **1.1. Habilitar Firestore en Firebase**

1. **Ve a la consola de Firebase.**
2. En el menú lateral, haz clic en **Firestore Database**.
3. Pulsa **Crear Base de Datos** y selecciona el **modo de prueba** para facilitar el desarrollo.

### **2.1. Configurar reglas de seguridad (Modo Desarrollo)**

En la pestaña **Reglas**, cambia a:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if true; // Permitir acceso sin restricciones (Solo para desarrollo)
    }
  }
}
```

**Recuerda modificar estas reglas antes de publicar la app en producción.**

---

## **3. Implementación de Firestore en Android**

Ahora pasamos a **crear las capas de nuestra arquitectura** siguiendo el **Repository Pattern**.

### **1.1. Crear FirebaseModule.kt para Inyección de Dependencias**

Para asegurarnos de que **Firestore** se inyecta correctamente en la app, creamos un **FirebaseModule**.

**Ubicación:** com.example.loginycardview.di.FirebaseModule.kt



```

package com.example.loginyocardview.di

import com.google.firebase.firestore.FirebaseFirestore
import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
object FirebaseModule {

    @Provides
    @Singleton
    fun provideFirestore(): FirebaseFirestore = FirebaseFirestore.getInstance()
}

```

Esto permite inyectar **Firestore** en nuestros Repositorios.

---

## 4. Crear el RepositoryModule.kt

Aquí es donde **inyectamos los repositorios**.

**Ubicación:** com.example.loginyocardview.di.RepositoryModule.kt

```

package com.example.loginyocardview.di

import com.example.loginyocardview.data.repository.EventRepositoryImpl
import com.example.loginyocardview.domain.EventRepository
import com.example.loginyocardview.data.repository.ProfessorRepositoryImpl
import com.example.loginyocardview.domain.ProfessorRepository
import com.example.loginyocardview.data.repository.VideoRepositoryImpl
import com.example.loginyocardview.domain.VideoRepository
import dagger.Binds
import dagger.Module
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindEventRepository(
        eventRepositoryImpl: EventRepositoryImpl
    ): EventRepository
}

```

```

@Binds
@Singleton
abstract fun bindProfessorRepository(
    professorRepositoryImpl: ProfessorRepositoryImpl
): ProfessorRepository

@Binds
@Singleton
abstract fun bindVideoRepository(
    videoRepositoryImpl: VideoRepositoryImpl
): VideoRepository
}

```

---

## **5. Implementar los Repositorios**

Aquí creamos los repositorios para interactuar con Firestore.

### **1.1.VideoRepositoryImpl.kt**

```

package com.example.loginycardview.data.repository

import android.util.Log
import com.example.loginycardview.domain.Video
import com.example.loginycardview.domain.VideoRepository
import com.google.firebase.firestore.FirebaseFirestore
import kotlinx.coroutines.tasks.await
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class VideoRepositoryImpl @Inject constructor(
    private val firestore: FirebaseFirestore
) : VideoRepository {

    override suspend fun getVideos(): List<Video> {
        return try {
            val snapshot = firestore.collection("videos").get().await()
            snapshot.documents.mapNotNull { it.toObject(Video::class.java) }
        } catch (e: Exception) {
            Log.e("FirestoreTest", "Error al obtener videos: ${e.message}", e)
            emptyList()
        }
    }

    override suspend fun saveVideo(video: Video) {
        try {
            firestore.collection("videos").add(video).await()
        } catch (e: Exception) {
            Log.e("FirestoreTest", "Error al guardar video: ${e.message}", e)
        }
    }
}

```

```
}
```

### 2.1.ProfessorRepositoryImpl.kt

```
package com.example.loginycardview.data.repository

import android.util.Log
import com.example.loginycardview.domain.Professor
import com.example.loginycardview.domain.ProfessorRepository
import com.google.firebase.firestore.FirebaseFirestore
import kotlinx.coroutines.tasks.await
import javax.inject.Inject
import javax.inject.Singleton

@Singleton
class ProfessorRepositoryImpl @Inject constructor(
    private val firestore: FirebaseFirestore
) : ProfessorRepository {

    override suspend fun getProfessors(): List<Professor> {
        return try {
            val snapshot = firestore.collection("professors").get().await()
            snapshot.documents.mapNotNull { it.toObject(Professor::class.java) }
        } catch (e: Exception) {
            Log.e("FirestoreTest", "Error al obtener profesores: ${e.message}", e)
            emptyList()
        }
    }

    override suspend fun saveProfessor(professor: Professor) {
        try {
            firestore.collection("professors").add(professor).await()
        } catch (e: Exception) {
            Log.e("FirestoreTest", "Error al guardar profesor: ${e.message}", e)
        }
    }
}
```

---

## 6. Conectar Firestore con ViewModel

Para que la UI pueda acceder a Firestore, usamos **ViewModel**.  
Ejemplo de VideoViewModel.kt:

```
package com.example.loginycardview.presentation.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelScope
import com.example.loginycardview.domain.Video
import com.example.loginycardview.domain.usecases.GetVideosUseCase
import com.example.loginycardview.domain.usecases.SaveVideoUseCase
```

```
import dagger.hilt.android.lifecycle.HiltViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
import javax.inject.Inject

@HiltViewModel
class VideoViewModel @Inject constructor(
    private val getVideosUseCase: GetVideosUseCase,
    private val saveVideoUseCase: SaveVideoUseCase
) : ViewModel() {

    private val _videos = MutableStateFlow<List<Video>>(emptyList())
    val videos: StateFlow<List<Video>> get() = _videos

    fun loadVideos() {
        viewModelScope.launch {
            _videos.value = getVideosUseCase()
        }
    }

    fun saveVideo(video: Video) {
        viewModelScope.launch {
            saveVideoUseCase(video)
            loadVideos()
        }
    }
}
```

---